

1. Diseño de pipelines reproducibles en Jupyter y Colab

En esta unidad, aprenderás a estructurar notebooks y entornos en Jupyter y Google Colab para crear pipelines de análisis reproducibles. Cubriremos las mejores prácticas para organizar el flujo de trabajo, documentar el proceso, gestionar dependencias y conectar almacenamiento en la nube, asegurando que tus análisis sean claros, replicables y colaborativos.

¿Alguna vez has intentado replicar un análisis de datos y te has encontrado con notebooks desordenados, dependencias rotas o resultados inconsistentes? En el mundo profesional de la ciencia de datos, la reproducibilidad es clave para garantizar que los análisis sean confiables, colaborativos y escalables. En esta unidad, exploraremos cómo diseñar pipelines reproducibles utilizando Jupyter y Google Colab, dos herramientas ampliamente usadas en la industria. Aprenderás a estructurar tus notebooks para que sean claros y fáciles de seguir, documentar cada paso del proceso, gestionar dependencias y conectar tu trabajo con almacenamiento en la nube para facilitar el acceso y la colaboración. Al finalizar, estarás preparado para crear flujos de trabajo que no solo funcionen en tu entorno, sino que puedan ser replicados por otros sin inconvenientes, un requisito fundamental en proyectos reales de ciencia de datos.

¿Qué es un pipeline reproducible?

Un *pipeline reproducible* es un flujo de trabajo estructurado que permite ejecutar un análisis de datos desde la adquisición hasta la visualización y almacenamiento, garantizando que cualquier persona pueda replicar los resultados con los mismos datos y código.



Estructura recomendada para notebooks reproducibles

Para lograr reproducibilidad, es fundamental organizar el notebook en secciones claras y coherentes:

- **Introducción y objetivos:** Explica el propósito del análisis.
- **Importación de librerías y configuración:** Incluye todas las dependencias y configuraciones iniciales.
- **Carga y limpieza de datos:** Procesos para preparar los datos.
- **Transformación y análisis:** Manipulación y exploración de datos.
- **Visualización:** Gráficos y representaciones visuales.
- **Guardado de resultados:** Exportación de artefactos como archivos CSV, imágenes o modelos.

Nota: Mantener cada sección con celdas pequeñas y comentadas facilita la comprensión y depuración.

Documentación efectiva

Utiliza títulos, subtítulos y comentarios claros. Emplea Markdown para explicar el contexto y las decisiones tomadas. Esto ayuda a otros (y a ti mismo en el futuro) a entender el flujo y la lógica del análisis.

Control de versiones ligero y gestión de dependencias en Colab

Aunque Colab no integra Git directamente, puedes:

- Usar Google Drive para almacenar versiones de notebooks.
- Documentar versiones de librerías con `!pip freeze`.
- Crear archivos `requirements.txt` para instalar dependencias específicas.

Ejemplo para instalar dependencias:

```
python
!pip install -r requirements.txt
```

Integración con almacenamiento en la nube

Google Colab permite montar Google Drive para acceder y guardar archivos:

```
python
from google.colab import drive
drive.mount('/content/drive')
```

Esto facilita compartir datasets, resultados y notebooks entre colaboradores.

Buenas prácticas para guardar y compartir artefactos

- Guarda resultados intermedios y finales en carpetas organizadas.
- Usa formatos estándar (CSV, PNG, Pickle).
- Documenta la ubicación y formato de cada archivo.

Con estos fundamentos, estarás listo para diseñar pipelines reproducibles que cumplan con estándares profesionales y faciliten la colaboración en proyectos de ciencia de datos.

En la práctica profesional, los proyectos de ciencia de datos suelen involucrar múltiples etapas y colaboradores. Un análisis reproducible asegura que cualquier miembro del equipo pueda entender,

ejecutar y extender el trabajo sin perder tiempo en resolver problemas de configuración o interpretación. Por ejemplo, en una empresa que analiza datos de ventas, un pipeline reproducible permite que el equipo de marketing actualice los análisis con nuevos datos sin errores, manteniendo la consistencia en reportes y decisiones.

Además, el uso de Google Colab y Google Drive facilita el trabajo remoto y la colaboración en tiempo real, eliminando barreras técnicas y mejorando la productividad. La gestión adecuada de dependencias y la documentación clara son prácticas que reducen riesgos y aceleran la entrega de resultados.

Esta unidad prepara el terreno para que puedas aplicar estas técnicas en tu proyecto práctico, integrando limpieza, transformación, visualización y almacenamiento en un flujo coherente y profesional.

2. Teoría y detección de valores ausentes (Missing Values)

Introducción a los valores ausentes en datasets, técnicas exploratorias para su detección y consideraciones básicas para decidir estrategias iniciales de manejo, preparando al estudiante para aplicar métodos avanzados en unidades posteriores.

¿Qué son los valores ausentes y por qué importan?

Imagina que estás analizando un conjunto de datos de clientes para una empresa de telecomunicaciones y descubres que muchas filas tienen información faltante en campos clave como edad o ingresos. ¿Cómo afecta esto a tus análisis y modelos predictivos? En esta unidad, exploraremos qué son los valores ausentes, cómo detectarlos eficazmente y qué impacto tienen en el análisis de datos. Aprenderás a utilizar técnicas exploratorias con Pandas para identificar patrones de datos faltantes y a tomar decisiones iniciales fundamentadas sobre cómo manejarlos, ya sea eliminándolos o imputándolos. Este conocimiento es fundamental para asegurar la calidad y confiabilidad de cualquier análisis de datos, y te preparará para aplicar técnicas avanzadas de imputación en las siguientes unidades.

Definición y tipos de valores ausentes

En el contexto de análisis de datos, un *valor ausente* es una observación que no tiene un valor registrado para una variable específica. En Python y Pandas, estos valores suelen representarse como NaN (Not a Number) o None.



Tipos comunes de valores ausentes:

- **MCAR (Missing Completely At Random):** La ausencia es completamente aleatoria y no depende de ninguna variable.
- **MAR (Missing At Random):** La ausencia depende de otras variables observadas.
- **MNAR (Missing Not At Random):** La ausencia depende del valor mismo que falta.

Nota: Entender el tipo de missingness es crucial para elegir la estrategia de manejo adecuada.

Impacto de valores ausentes en análisis

Los valores ausentes pueden:

- Sesgar resultados estadísticos y modelos predictivos.
- Reducir la cantidad de datos disponibles si se eliminan filas o columnas.
- Afectar la visualización y comprensión de los datos.

Técnicas exploratorias para detección

Conteo de valores ausentes

Pandas ofrece métodos sencillos para detectar valores ausentes:

```
python
import pandas as pd
df = pd.read_csv('datos.csv')
# Conteo total de valores ausentes por columna

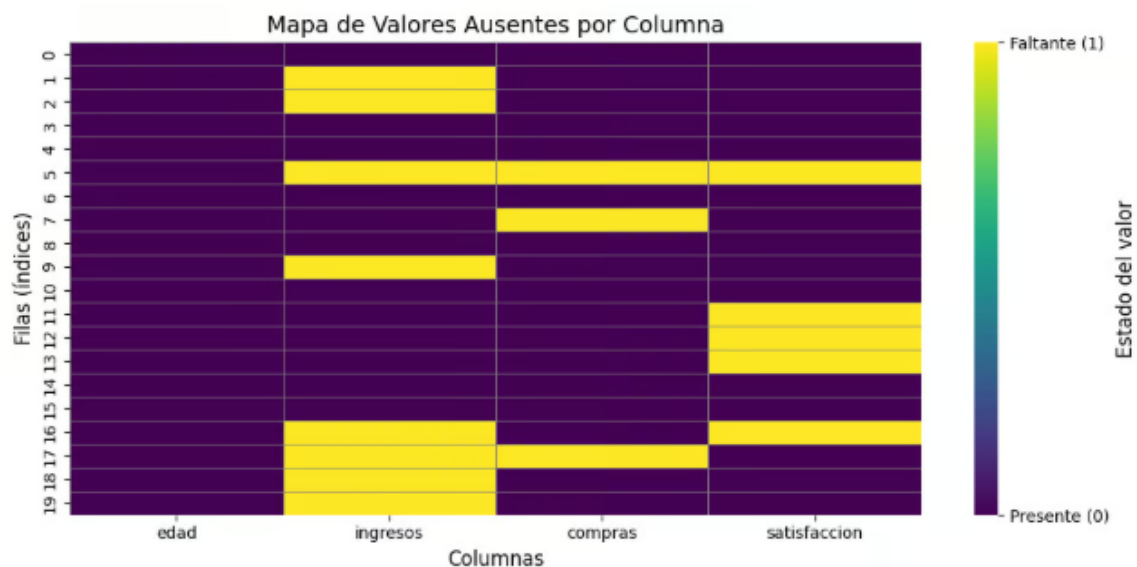
missing_counts = df.isna().sum()
print(missing_counts)
# Porcentaje de valores ausentes
missing_percent = df.isna().mean() * 100
print(missing_percent)
```

Visualización con heatmaps

La librería seaborn permite visualizar patrones de missingness:

```
python
import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(df.isna(), cbar=False, yticklabels=False)
plt.title('Mapa de valores ausentes')
plt.show()
```

Esta visualización ayuda a identificar si los valores ausentes están concentrados en ciertas filas o columnas.



Patrones de missingness

Analizar si los valores ausentes ocurren en patrones específicos puede indicar problemas en la recolección de datos o sesgos.

Estrategias iniciales para manejar valores ausentes

- **Eliminación:** Usar `dropna()` para eliminar filas o columnas con valores ausentes. Adecuado cuando la cantidad de datos faltantes es pequeña y no sesga el análisis.
- **Relleno simple:** Usar `fillna()` para reemplazar valores ausentes con un valor constante, la media, mediana o moda.

Ejemplo:

python

```
df['edad'] = df['edad'].fillna(df['edad'].median())
```

Consideraciones estadísticas básicas

- Eliminar datos puede reducir la representatividad.
- Rellenar con valores constantes puede introducir sesgos.
- La elección depende del contexto y del tipo de missingness.

Estrategias intermedias

Análisis del patrón de valores ausentes

Antes de imputar, es fundamental identificar:

- qué variables presentan missing
- cuántos valores faltan
- si existen patrones sistemáticos

Ejemplos de preguntas clave:

- ¿faltan más datos en ciertos períodos?
- ¿faltan datos solo para determinados grupos?
- ¿hay correlación entre variables ausentes?

Herramientas habituales:

```
df.isna().mean().sort_values(ascending=False)
```

Esto permite priorizar variables críticas y detectar columnas problemáticas.

Eliminación condicionada

En lugar de eliminar filas de forma global, se puede:

- eliminar solo cuando falten variables clave
- conservar registros parcialmente incompletos

Ejemplo conceptual:

- eliminar si faltan **precio y superficie**
- conservar si falta una variable secundaria

Este enfoque mantiene volumen de datos sin comprometer el análisis central.

Imputación por grupo (group-based imputation)

Los valores faltantes pueden imputarse según segmentos homogéneos:

- por categoría
- por zona
- por tipo de cliente
- por producto

Ejemplo:

```
df['precio'] = df['precio'].fillna(  
df.groupby('categoria')['precio'].transform('median'))
```

Ventaja clave:

respeta la estructura interna del dataset.

Uso de estadísticas robustas

Cuando existen outliers:

- la media puede ser engañosa

- la mediana o percentiles son más estables

Ejemplo:

```
df['ingreso'] = df['ingreso'].fillna(df['ingreso'].quantile(0.5))
```

Este enfoque es especialmente útil en variables económicas o financieras.

Estrategias avanzadas

5. Comprensión del tipo de missingness

En Data Science se distinguen tres tipos fundamentales:

Tipo	Descripción
MCAR	Missing Completely At Random
MAR	Missing At Random (depende de otras variables)
MNAR	Missing Not At Random (depende del valor faltante)

La técnica de imputación debe elegirse según este diagnóstico.

Ejemplo:

- ingresos faltantes solo en personas de altos cargos → MNAR

Variables indicadoras de missing

En muchos modelos, el hecho de que un valor falte es información.

Estrategia:

- crear una columna binaria indicando ausencia

```
df['edad_missing'] = df['edad'].isna().astype(int)
```

Esto permite al modelo aprender patrones ocultos.

Muy utilizado en:

- modelos de crédito

- churn
- fraude
- scoring

Imputación múltiple

Técnica estadística avanzada:

- se generan múltiples datasets imputados
- se entrena el modelo en cada uno
- se promedian los resultados

Beneficio:

- incorpora incertidumbre
- reduce sobreconfianza del modelo

Usada en:

- estadística aplicada
- investigación científica
- economía

Enfoque profesional

En proyectos reales:

No existe una única forma correcta de imputar valores faltantes.

Existe una forma justificada.

La calidad del análisis no se mide por “rellenar nulos”, sino por:

- comprensión del negocio
- análisis del patrón de ausencia

- impacto estadístico
- coherencia con el modelo final

Las técnicas básicas permiten limpiar datos.

Las técnicas intermedias permiten preservar información.

Las técnicas avanzadas permiten modelar la incertidumbre.

Dominar el tratamiento de valores ausentes es uno de los puntos que separa:

- un análisis exploratorio
- de un pipeline profesional de Data Science

Con este conocimiento, estarás listo para explorar técnicas más avanzadas de imputación y preparar tus datos para análisis temporales y modelado predictivo.

En la práctica profesional de ciencia de datos, los valores ausentes son una realidad común en datasets provenientes de encuestas, sensores, registros transaccionales o bases de datos empresariales. Detectarlos y entender su naturaleza es el primer paso para garantizar análisis confiables.

Por ejemplo, en un proyecto de análisis de churn para una empresa de telecomunicaciones, ignorar o manejar incorrectamente valores ausentes en variables como duración de llamadas o ingresos puede llevar a modelos inexactos y decisiones erróneas.

Las técnicas exploratorias que hemos visto permiten a los analistas identificar rápidamente la extensión y patrones de missingness, facilitando la selección de estrategias adecuadas para el preprocesamiento.

Este enfoque práctico y fundamentado es esencial para preparar los datos antes de aplicar técnicas avanzadas de imputación con Scikit-Learn, que veremos en las siguientes unidades. Además, una correcta gestión de valores ausentes es clave para el análisis temporal y la visualización efectiva, temas que abordaremos más adelante en el módulo.

3. Estrategias de Imputación (Básicas y Avanzadas)

Ejercicio para reforzar el manejo básico de valores ausentes en pandas mediante técnicas simples de imputación como dropna y fillna, aplicando transformaciones reproducibles en notebooks.

En este ejercicio, reforzaremos el manejo básico de valores ausentes en pandas, aplicando técnicas simples de imputación como dropna y fillna. Aprenderás a eliminar filas con valores ausentes, rellenar con constantes o valores derivados de grupos, y registrar estas transformaciones para asegurar la reproducibilidad en tus notebooks.

Descripción del problema

Se te proporciona un dataset tabular con valores ausentes (NaN). Tu tarea es implementar una función que reciba este dataset y una serie de instrucciones para imputar los valores ausentes siguiendo estas reglas:

- Eliminar filas que tengan valores ausentes en columnas específicas.
- Rellenar valores ausentes en columnas numéricas con una constante dada.
- Rellenar valores ausentes en columnas categóricas con el valor más frecuente (moda) de esa columna.
- Rellenar valores ausentes en columnas numéricas agrupando por otra columna y usando el valor medio del grupo.

Además, debes imprimir el dataset resultante tras aplicar las imputaciones.

Formato de entrada

- La primera línea contiene dos enteros `n` y `m` separados por espacio: número de filas y columnas del dataset.
- La siguiente línea contiene `m` nombres de columnas separados por espacio.
- Las siguientes `n` líneas contienen los datos de cada fila, con valores separados por espacio. Los valores ausentes se representan con la cadena `NA`.
- La siguiente línea contiene un entero `k` con el número de instrucciones de imputación.
- Las siguientes `k` líneas contienen instrucciones en uno de los siguientes formatos:

1. `dropna col_name`

Eliminar filas con valores ausentes en la columna `col_name`.

2. `fillna_const col_name value`

Rellenar valores ausentes en la columna `col_name` con la constante `value`.

3. `fillna_mode col_name`

Rellenar valores ausentes en la columna `col_name` con la moda (valor más frecuente) de esa columna.

4. `fillna_group_mean col_name group_col`

Rellenar valores ausentes en la columna `col_name` con la media del grupo definido por `group_col`.

Formato de salida

- Imprime el dataset resultante tras aplicar todas las instrucciones, con los valores separados por espacio.
 - Los valores numéricos deben imprimirse con dos decimales.
 - Los valores ausentes que no hayan sido imputados deben imprimirse como `NA`.
-

Ejemplo

Entrada

```
5 3
age gender income
25 M 50000
NA F 60000
30 NA NA
22 M NA
NA F 45000
3
dropna gender
fillna_const income 55000
fillna_mode age
```

Salida

```
25.00 M 50000.00
30.00 F 60000.00
22.00 M 55000.00
NA F 45000.00
```

Notas

- La función debe manejar correctamente los tipos de datos: numéricos y categóricos.
- La reproducibilidad implica que las transformaciones se apliquen en el orden dado.
- Se recomienda usar `pandas` para facilitar la manipulación.

4. Imputación avanzada con Scikit-Learn y Pipelines

Ejercicio para practicar la configuración y uso de imputadores avanzados (SimpleImputer e IterativeImputer) integrados en pipelines y ColumnTransformer de Scikit-Learn para un flujo reproducible de preprocesamiento de datos con valores ausentes.

¿Qué es scikit-learn y para qué sirve?

scikit-learn es una de las librerías más usadas en Python para machine learning clásico (regresión, clasificación, clustering, reducción de dimensión, preprocesamiento). Sus ventajas para principiantes:

- API consistente: `fit()`, `transform()`, `predict()/score()` en la mayoría de los objetos.
- Muchas herramientas integradas para preprocesado (escalado, codificación, imputación), modelos y evaluación (cross-validation, métricas).
- Fácil integración en pipelines reproducibles (Pipeline, ColumnTransformer).

Conceptos clave para principiantes:

- **Estimador:** cualquier objeto con `fit(X, y)` (ej.: `LinearRegression`, `SimpleImputer`).
 - **Transformador:** estimador que implementa `transform(X)` (ej.: `StandardScaler`, `OneHotEncoder`).
 - **Pipeline:** encadena transformadores y un estimador final para evitar fugas de datos (data leakage).
 - **ColumnTransformer:** aplica transformaciones diferentes por columnas (numéricas vs categóricas).
-

¿Por qué imputar datos (qué problema resuelve)?

Muchas tablas reales tienen valores faltantes. Los modelos de scikit-learn no aceptan NaN en entradas; además, ignorar filas con NaN puede sesgar tus resultados o reducir la muestra. La imputación reemplaza valores faltantes con estimaciones basadas en la información disponible.

Tipos de ausencia:

- **MCAR (Missing Completely At Random):** ausencia no relacionada con datos observados ni no observados.
- **MAR (Missing At Random):** ausencia relacionada con datos observados.
- **MNAR (Missing Not At Random):** ausencia relacionada con el valor que falta (más difícil de manejar).

Estrategia de imputación depende del mecanismo y del contexto: medios simples (media/mediana/moda) o métodos multivariantes más complejos (regresión iterativa, modelos basados en árboles).

`SimpleImputer`:

`SimpleImputer` es la forma más básica y robusta para empezar.

¿Qué hace?

Reemplaza `missing_values` (por defecto `np.nan`) por:

- `strategy='mean'` — media (numérica).
- `strategy='median'` — mediana (robusta ante outliers).
- `strategy='most_frequent'` — moda (útil para categóricas).
- `strategy='constant'` — un valor fijo (ej. `'missing'` o `0`).

Ventajas

- Rápido, fácil de entender e implementar.
- Funciona bien cuando la relación entre variables no es compleja.
- Útil como baseline o paso dentro de pipelines.

Limitaciones

- Ignora correlaciones entre columnas (imputa cada columna por separado).
- Puede introducir sesgos si los datos no son MCAR.
- No captura relaciones multivariantes (p. ej. si age y income están relacionadas).

Parámetros relevantes

- `missing_values`: valor a considerar como faltante (por defecto `np.nan`).
 - `strategy`: `'mean'|'median'|'most_frequent'|'constant'`.
 - `fill_value`: si `strategy='constant'`, valor usado.
 - `add_indicator=True`: opcionalmente añade una columna indicadora de faltantes (útil para que el modelo sepa dónde hubo imputación).
-

Imputación multivariante: IterativeImputer (breve teoría)

IterativeImputer realiza imputación multivariante por regresión iterativa (MICE-style): cada variable con NaN se modela como función de las otras, iterando hasta convergencia. Es más potente que SimpleImputer cuando hay relaciones entre columnas.

- Pros: captura dependencias entre features; a menudo produce imputaciones más realistas.
- Contras: más costoso computacionalmente; hay que cuidar fuga de información si no se aplica dentro de un pipeline correctamente (si se usa en CV, debe ajustarse sólo en training).

Nota práctica: en versiones antiguas de scikit-learn había que habilitar como experimental (from sklearn.experimental import enable_iterative_imputer), en versiones recientes suele estar estable. Incluiré la import compatible abajo.

Contexto

En análisis de datos y machine learning, es común encontrar valores ausentes. La imputación es una técnica para reemplazar estos valores con estimaciones razonables, permitiendo que los modelos funcionen correctamente.

Scikit-Learn ofrece herramientas para imputar datos y combinarlas con transformaciones y modelos en pipelines, facilitando la reproducibilidad y mantenimiento del código.

Te pasamos un ejercicio completo para que copies y pegues en tu entorno de desarrollo y puedas ver paso a paso como funciona esta estrategia de imputación de valores faltantes, cuya función va a servir para dejar los datasets preparados para el futuro modelado en Machine Learning.

¿Qué está pasando?

`1 fit()`

El imputador aprende:

- La mediana de cada columna numérica
- Las categorías existentes
- (En IterativeImputer) las relaciones entre variables

2 transform()

Reemplaza los NaN por:

- Mediana (SimpleImputer)
- Valor estimado por modelo iterativo (IterativeImputer)
- "missing" en categóricas

3 ColumnTransformer

Permite aplicar:

- Un tratamiento a numéricas
- Otro distinto a categóricas

En un mismo flujo reproducible.

Diferencia conceptual clave

SimpleImputer	IterativeImputer
Univariante	Multivariante
Cada columna se imputa sola	Usa relación entre variables
Muy rápido	Más costoso
Baseline robusto	Más realista si hay correlaciones

Notas

- Usa SimpleImputer(strategy='most_frequent') para columnas categóricas.
- Usa IterativeImputer(random_state=0) para columnas numéricas.
- Asegúrate que el DataFrame resultante tenga las columnas en el mismo orden que el original.
- No modifiques el orden de las filas.

5. Parsing de fechas e indexado temporal

En esta unidad, aprenderás a convertir columnas de datos en formatos de fecha y hora utilizando Pandas, crear índices temporales para facilitar el análisis y realizar operaciones de slicing para filtrar datos por rangos de fechas. Abordaremos problemas comunes al parsear fechas, formatos mixtos y estrategias para normalizar timestamps, preparando tus series temporales para análisis y visualización posteriores.

Introducción al parsing de fechas en series temporales

Imagina que tienes un conjunto de datos con registros de ventas que incluyen fechas en formatos variados: algunas en dd/mm/yyyy, otras en yyyy-mm-dd, y algunas con horas y minutos. ¿Cómo puedes analizar tendencias temporales si las fechas no están uniformemente formateadas? Esta unidad te guiará para convertir esas columnas en formatos de fecha y hora estándar usando Pandas, crear índices temporales que faciliten el análisis y aplicar técnicas de slicing para filtrar datos por períodos específicos.

Al finalizar, serás capaz de transformar datos crudos con fechas inconsistentes en series temporales limpias y estructuradas, una habilidad esencial para cualquier científico de datos que trabaje con análisis temporal en la industria.

Conceptos clave: Parsing e indexado temporal con Pandas



¿Qué es el parsing de fechas?

El *parsing* de fechas es el proceso de convertir datos en formato texto o numérico que representan fechas y horas en objetos de tipo `datetime` que Pandas puede interpretar y manipular.

Conversión a `datetime`

Pandas ofrece la función `pd.to_datetime()` para convertir columnas o series a objetos `datetime`:

```
python
```

```
import pandas as pd
df['fecha'] = pd.to_datetime(df['fecha'], errors='coerce')
```

- `errors='coerce'` convierte valores inválidos en NaT (Not a Time), facilitando la limpieza.

Creación de índices temporales

Una vez convertida la columna, podemos establecerla como índice para aprovechar funcionalidades de series temporales:

```
python
df.set_index('fecha', inplace=True)
```

Esto permite realizar operaciones eficientes de slicing y resampling.

Slicing y filtrado por fechas

Con un índice temporal, podemos filtrar datos por rangos usando slicing:

```
python
# Filtrar datos entre dos fechas
filtro = df.loc['2023-01-01':'2023-01-31']
```

También es posible filtrar por año, mes o día:

```
python
enero = df.loc['2023-01']
```

Manejo de formatos mixtos y problemas comunes

- Fechas en formatos mixtos pueden requerir parámetros adicionales en `pd.to_datetime()`, como `format` o `dayfirst`.
- Valores nulos o erróneos deben ser detectados y tratados.

Nota: Siempre verifica la conversión con `df['fecha'].dtype` y muestra muestras para confirmar el parsing correcto.

Ejemplo práctico:

Supongamos un DataFrame con fechas en formatos mixtos:

```
python
import pandas as pd
data = {'fecha': ['01/02/2023', '2023-03-15', '15-04-2023', '2023/05/20'],
        'valor': [10, 20, 15, 30]}
df = pd.DataFrame(data)

df['fecha'] = pd.to_datetime(df['fecha'], dayfirst=True, errors='coerce')
df.set_index('fecha', inplace=True)
print(df)
```

Esto normaliza las fechas y permite análisis temporal consistente.

Aplicación práctica en análisis de datos temporales

En la industria, los datos temporales son omnipresentes: ventas diarias, registros de sensores, logs de sistemas, entre otros. Sin un parsing correcto, cualquier análisis de tendencias, estacionalidad o anomalías será erróneo.

Por ejemplo, en un proyecto de análisis de ventas, convertir correctamente las fechas permite:

- Agrupar ventas por día, semana o mes
- Detectar patrones estacionales
- Filtrar períodos específicos para campañas o eventos

Además, establecer un índice temporal facilita la integración con técnicas avanzadas como resampling y rolling windows, que veremos en unidades posteriores.

Preparar tus datos con un parsing y indexado correcto es el primer paso para un análisis temporal robusto y confiable.

6 - Zonas horarias, shift y frecuencia

Ejercicio para practicar la conversión entre zonas horarias, el uso de shift para crear features lag y la manipulación de frecuencias temporales en series de tiempo.

Ejercicio: Manipulación de series temporales con zonas horarias, shift y frecuencia

1. Introducción

En los problemas reales de negocio, una gran parte de la información se registra como series temporales, es decir, conjuntos de observaciones ordenadas cronológicamente. Ejemplos frecuentes incluyen:

- ventas por hora
- transacciones financieras

- eventos de sistemas
- métricas de uso de aplicaciones
- registros de sensores

El análisis correcto de este tipo de datos requiere comprender no solo los valores, sino también el tiempo asociado a cada observación.

En esta práctica se trabaja con una serie temporal horaria que representa una métrica de negocio registrada en UTC, el estándar internacional de tiempo. A partir de esta serie se aplican tres transformaciones fundamentales utilizadas en Data Science:

1. Conversión de zonas horarias
2. Desplazamiento temporal (*lag*)
3. Cambio de frecuencia (*resampling*)

Estas operaciones son esenciales para garantizar análisis coherentes, comparaciones correctas y agregaciones consistentes.

2. Serie temporal y `DateTimeIndex`

En pandas, una serie temporal se representa mediante un objeto `Series` o `DataFrame` cuyo índice es de tipo:

`DateTimeIndex`

Este tipo de índice permite:

- ordenar observaciones temporalmente
- realizar agregaciones por día, semana o mes
- desplazar valores en el tiempo
- trabajar con zonas horarias

En esta práctica, la serie se construye con timestamps tz-aware, es decir, con información explícita de zona horaria (UTC). Esto es un requisito indispensable para poder realizar conversiones horarias correctas.

3. Conversión de zonas horarias

3.1. ¿Por qué es importante?

En contextos reales, los sistemas suelen registrar eventos en UTC porque:

- evita ambigüedades internacionales
- no depende del horario local
- es estándar en bases de datos y APIs

Sin embargo, los analistas y equipos de negocio suelen trabajar con horarios locales, por ejemplo:

- hora comercial
- horarios de atención
- comportamiento de clientes según franja horaria

Por este motivo, resulta fundamental convertir la serie a la zona correspondiente al contexto de análisis.

3.2. `tz_localize` vs `tz_convert`

Es importante distinguir dos conceptos:

Método	Uso
<code>tz_localize()</code>	Asigna una zona horaria a datos que no la tienen
<code>tz_convert()</code>	Convierte entre zonas horarias ya definidas

En esta práctica **NO se debe usar** `tz_localize`, ya que los timestamps ya están en UTC.

El procedimiento correcto es:

```
serie_ny = serie_utc.tz_convert("America/New_York")
```

Esta operación:

- mantiene el orden temporal
 - modifica las horas según la diferencia horaria
 - conserva la correspondencia real de los eventos
-

4. Desplazamiento temporal (shift / lag)

4.1. ¿Qué es un lag?

Un *lag* representa el valor de una variable en un instante anterior.

Por ejemplo:

- ventas de la hora anterior
- consumo del día previo
- tráfico del período anterior

Este concepto es central en:

- análisis temporal
- modelos predictivos
- detección de cambios
- features para machine learning

4.2. Funcionamiento de `shift()`

El método:

```
serie.shift(1)
```

realiza las siguientes acciones:

- desplaza los valores una posición hacia abajo
- el índice permanece intacto
- el primer valor queda como NaN

Formalmente:

$$x_{t-lag1} = x_{t-1} \quad x_{t-1} = x_{t-lag1}$$

Esto permite comparar el valor actual con el pasado inmediato sin perder alineación temporal.

4.3. Interpretación analítica

Luego del desplazamiento:

- cada timestamp conserva su fecha y hora
- el valor representa el período anterior
- se habilita análisis como:
- crecimiento
- diferencias
- variaciones intertemporales

Este tipo de transformación es la base de muchos modelos de series temporales.

5. Cambio de frecuencia (resample)

5.1. ¿Qué es el resampling?

El *resampling* consiste en **cambiar la granularidad temporal** de una serie.

Ejemplos:

- de horas → días
- de días → semanas
- de minutos → horas

En términos de negocio, esto permite responder preguntas como:

- ¿cuánto se vendió por día?
 - ¿cuál fue el total semanal?
 - ¿cómo se comportan los volúmenes agregados?
-

5.2. Uso de `resample()`

El método general es:

```
serie.resample("D").sum()
```

Donde:

- "D" indica frecuencia diaria
- `sum()` define cómo se agregan los valores

Pandas agrupa automáticamente todas las observaciones cuya marca horaria pertenece al mismo día calendario según la **zona horaria actual del índice**.

Esto es clave:

el resampling depende directamente de la zona horaria.

5.3. Importancia de la zona horaria en el resampling

Si la serie estuviera en UTC:

- los días se agruparían según UTC

Si está en America/New_York:

- los días se agrupan según horario local

Esto puede generar diferencias reales en reportes de negocio, por lo que el orden correcto de operaciones es:

1. convertir zona horaria
 2. luego aplicar resampling
-

6. Secuencia correcta de la práctica

La práctica sigue una lógica profesional real:

1. **Datos crudos en UTC**
2. **Conversión a horario local**
3. **Análisis temporal mediante lag**
4. **Agregación diaria para reporting**

Esta secuencia refleja cómo se trabajan las series temporales en:

- análisis operativo
 - dashboards
 - pipelines analíticos
 - feature engineering
-

7. Conclusión

El manejo correcto del tiempo es uno de los aspectos más críticos —y más frecuentemente mal implementados— en proyectos de Data Science.

Esta práctica permite comprender de forma controlada:

- cómo pandas representa el tiempo
- cómo se transforman las series temporales
- cómo se alinean datos históricos
- cómo se construyen métricas agregadas

Estos conceptos constituyen la base para:

- análisis de tendencias
- forecasting
- detección de anomalías
- construcción de variables temporales

y son indispensables en cualquier entorno profesional que trabaje con datos reales.

Te pasamos un ejercicio completo para que copies y pegues en tu entorno de desarrollo y puedas ver paso a paso como funciona

Nota: Usa pandas para manipular las series temporales y asegúrate de manejar correctamente las zonas horarias y los NaN en el desplazamiento.

7 - Resampling y ventanas móviles (rolling)

Ejercicio para aplicar técnicas de resampling y cálculo de estadísticas móviles en series temporales. Se trabajará con datos de ventas diarias para obtener agregaciones semanales y medias móviles, interpretando los resultados para análisis de tendencias.

Ejercicio: Manipulación de series temporales con zonas horarias, shift y frecuencia

1. Introducción

En los problemas reales de negocio, una gran parte de la información se registra como series temporales, es decir, conjuntos de observaciones ordenadas cronológicamente.

Ejemplos frecuentes incluyen:

- ventas por hora
- transacciones financieras
- eventos de sistemas
- métricas de uso de aplicaciones
- registros de sensores

El análisis correcto de este tipo de datos requiere comprender no solo los valores, sino también el tiempo asociado a cada observación.

En esta práctica se trabaja con una serie temporal horaria que representa una métrica de negocio registrada en UTC, el estándar internacional de tiempo. A partir de esta serie se aplican tres transformaciones fundamentales utilizadas en Data Science:

1. Conversión de zonas horarias
2. Desplazamiento temporal (*lag*)
3. Cambio de frecuencia (*resampling*)

Estas operaciones son esenciales para garantizar análisis coherentes, comparaciones correctas y agregaciones consistentes.

2. Serie temporal y DateTimeIndex

En pandas, una serie temporal se representa mediante un objeto `Series` o `DataFrame` cuyo índice es de tipo: `DateTimeIndex`

Este tipo de índice permite:

- ordenar observaciones temporalmente
- realizar agregaciones por día, semana o mes
- desplazar valores en el tiempo
- trabajar con zonas horarias

7 — Resampling y ventanas móviles (rolling)

¿Por qué son útiles estas técnicas?

En series temporales cada observación tiene una **marca temporal**. Dos técnicas fundamentales para analizarlas son:

- **Resampling:** cambiar la frecuencia de la serie (ej. de diario → semanal, de minuto → hora). Sirve para agregar ruido, ver tendencias a más largo plazo, o alinear series con distintas frecuencias.
- **Ventanas móviles (rolling):** calcular estadísticas locales (media móvil, desviación, suma acumulada en ventana) que “suavizan” la serie y revelan tendencia/estacionalidad locales.

Ambas técnicas se usan muchísimo en negocio: smoothing de ventas, cálculo de promedios móviles para detección de anomalías, agregación por periodos contables, etc.

Conceptos clave

- **Frecuencia:** 'D' (diaria), 'W' (semanal), 'M' (mensual), 'H' (horaria), etc.
 - **Resampling (downsample / upsample):**
 - *Downsample* (reduce resolución): agregas (sum, mean).
 - *Upsample* (aumenta resolución): introduces nuevos índices, debes rellenar (ffill, bfill, interpolate).
 - **Window size:** número de observaciones en la ventana (ej. `window=7` para 7 días).
 - **Center vs right/left:** si la media móvil se asocia al centro de la ventana o al límite derecho.
 - **min_periods:** mínimos valores no nulos para devolver resultado.
 - **Expanding:** ventanas que crecen desde el inicio (acumulado).
 - **EWMA:** media móvil exponencial (pondera más reciente).
-

Conversión de zonas horarias

¿Por qué es importante?

En contextos reales, los sistemas suelen registrar eventos en UTC porque:

- evita ambigüedades internacionales
- no depende del horario local
- es estándar en bases de datos y APIs

Sin embargo, los analistas y equipos de negocio suelen trabajar con **horarios locales**, por ejemplo:

- hora comercial
- horarios de atención
- comportamiento de clientes según franja horaria

Por este motivo, resulta fundamental convertir la serie a la zona correspondiente al contexto de análisis.

`tz_localize` vs `tz_convert`

Es importante distinguir dos conceptos:

Método	Uso
<code>tz_localize()</code>	Asigna una zona horaria a datos que no la tienen
<code>tz_convert()</code>	Convierte entre zonas horarias ya definidas

En esta práctica **NO se debe usar** `tz_localize`, ya que los timestamps ya están en UTC.

El procedimiento correcto es:

```
serie_ny = serie_utc.tz_convert("America/New_York")
```

Esta operación:

- mantiene el orden temporal
 - modifica las horas según la diferencia horaria
 - conserva la correspondencia real de los eventos
-

Desplazamiento temporal (shift / lag)

¿Qué es un lag?

Un *lag* representa el valor de una variable en un instante anterior.

Por ejemplo:

- ventas de la hora anterior
- consumo del día previo
- tráfico del período anterior

Este concepto es central en:

- análisis temporal
 - modelos predictivos
 - detección de cambios
 - features para machine learning
-

Funcionamiento de `shift()`

El método:

```
serie.shift(1)
```

realiza las siguientes acciones:

- desplaza los valores una posición hacia abajo
- el índice permanece intacto
- el primer valor queda como NaN

Formalmente:

$$x_{t-lag1} = x_{t-1} \quad x_{t-lag1} = x_{t-1}$$

Esto permite comparar el valor actual con el pasado inmediato sin perder alineación temporal.

Interpretación analítica

Luego del desplazamiento:

- cada timestamp conserva su fecha y hora
- el valor representa el período anterior
- se habilita análisis como:
- crecimiento
- diferencias
- variaciones intertemporales

Este tipo de transformación es la base de muchos modelos de series temporales.

Cambio de frecuencia (resample)

¿Qué es el resampling?

El *resampling* consiste en **cambiar la granularidad temporal** de una serie.

Ejemplos:

- de horas → días
- de días → semanas
- de minutos → horas

En términos de negocio, esto permite responder preguntas como:

- ¿cuánto se vendió por día?
- ¿cuál fue el total semanal?
- ¿cómo se comportan los volúmenes agregados?

Uso de `resample()`

El método general es:

```
serie.resample("D").sum()
```

Donde:

- `"D"` indica frecuencia diaria
 - `sum()` define cómo se agregan los valores
- Pandas agrupa automáticamente todas las observaciones cuya marca horaria pertenece al mismo día calendario según la zona horaria actual del índice.
- Esto es clave:
el resampling depende directamente de la zona horaria.

Importancia de la zona horaria en el resampling

Si la serie estuviera en UTC:

- los días se agruparían según UTC

Si está en America/New_York:

- los días se agrupan según horario local

Esto puede generar diferencias reales en reportes de negocio, por lo que el orden correcto de operaciones es:

1. convertir zona horaria
2. luego aplicar resampling

Secuencia correcta de la práctica

La práctica sigue una lógica profesional real:

1. **Datos crudos en UTC**
2. **Conversión a horario local**
3. **Análisis temporal mediante lag**
4. **Agregación diaria para reporting**

Esta secuencia refleja cómo se trabajan las series temporales en:

- análisis operativo
- dashboards
- pipelines analíticos
- feature engineering

Conclusión

El manejo correcto del tiempo es uno de los aspectos más críticos —y más frecuentemente mal implementados— en proyectos de Data Science.

Esta práctica permite comprender de forma controlada:

- cómo pandas representa el tiempo
- cómo se transforman las series temporales
- cómo se alinean datos históricos
- cómo se construyen métricas agregadas

Estos conceptos constituyen la base para:

- análisis de tendencias
- forecasting

- detección de anomalías
- construcción de variables temporales

y son indispensables en cualquier entorno profesional que trabaje con datos reales.

Te pasamos un ejercicio completo para que copies y pegues en tu entorno de desarrollo y puedas ver paso a paso como funciona

Nota: Usa pandas para manipular las series temporales y asegúrate de manejar correctamente las zonas horarias y los NaN en el desplazamiento.

8 - Introducción a Pandas y NumPy: estructura y buenas prácticas

Este módulo introduce a los estudiantes en las bases fundamentales de NumPy y Pandas, dos librerías esenciales para la manipulación eficiente de datos en Python. Se explicará la relación entre ambas, las estructuras de datos clave como Series y DataFrame, y se mostrarán buenas prácticas para la selección y filtrado de datos. Además, se abordará la configuración del entorno de trabajo en Jupyter Notebook o Google Colab, y la lectura básica de archivos CSV, con especial atención a los tipos de datos y su impacto en el rendimiento y uso de memoria.

Introducción

Imagina que tienes un conjunto de datos con miles o incluso millones de registros, y necesitas analizarlos rápidamente para tomar decisiones informadas. ¿Cómo puedes manejar y transformar esta información de manera eficiente y profesional? Aquí es donde entran en juego NumPy y Pandas, dos librerías fundamentales en el ecosistema de Python para ciencia de datos.

En esta unidad, exploraremos cómo estas herramientas trabajan juntas para facilitar la manipulación de datos, desde la estructura básica de los arrays de NumPy hasta las poderosas estructuras de Pandas como Series y DataFrames. También aprenderás a configurar tu entorno de trabajo en Jupyter Notebook o Google Colab, leer archivos CSV y aplicar técnicas básicas de selección y filtrado de datos.

Al finalizar, serás capaz de entender la relación entre NumPy y Pandas, identificar sus estructuras clave y aplicar buenas prácticas para manejar datos de forma eficiente, sentando las bases para análisis más complejos en las siguientes unidades.

Fundamentos de NumPy para manipulación eficiente

NumPy es la librería base para computación numérica en Python. Su estructura principal es el array, que permite almacenar datos en forma de matrices multidimensionales con un tipo de dato homogéneo, lo que optimiza el uso de memoria y la velocidad de procesamiento.

Arrays y operaciones element-wise

Los arrays de NumPy permiten realizar operaciones matemáticas y lógicas de forma vectorizada, es decir, aplicando la operación a todos los elementos sin necesidad de bucles explícitos. Por ejemplo:

python

```
import numpy as np
arr = np.array([1, 2, 3])arr2 = arr * 2 # Multiplica cada elemento por 2
```

Broadcasting

El broadcasting es una característica que permite realizar operaciones entre arrays de diferentes formas, adaptando automáticamente las dimensiones para que sean compatibles. Esto es fundamental para escribir código eficiente y legible.

Conversión entre NumPy y Pandas

Pandas se construye sobre NumPy y utiliza sus arrays internamente. Es común convertir entre estructuras de Pandas y NumPy para aprovechar operaciones de alto rendimiento:

python

```
import pandas as pd
s = pd.Series([1, 2, 3])arr = s.values # Convertir Series a array de NumPy
```

Introducción a Pandas: Series y DataFrame

Pandas ofrece dos estructuras principales:

- **Series:** arreglo unidimensional etiquetado, similar a un array con índices personalizados.
 - **DataFrame:** tabla bidimensional con filas y columnas etiquetadas, ideal para datos heterogéneos.
- Estas estructuras facilitan la manipulación y análisis de datos tabulares.

Tipos de datos y memoria

Cada columna en un DataFrame tiene un tipo de dato (dtype) que afecta el uso de memoria y el rendimiento. Por ejemplo, usar int64 cuando un int8 es suficiente puede desperdiciar recursos. Pandas permite inspeccionar y optimizar estos tipos para mejorar la eficiencia.

Selección y filtrado de DataFrames

Pandas ofrece múltiples técnicas para seleccionar y filtrar datos:

- Selección por etiquetas con `.loc[]`
- Selección por posición con `.iloc[]`
- Filtrado booleano con condiciones
- Uso de máscaras y condiciones compuestas

Ejemplo básico:

python

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# Seleccionar columna 'A' col_a = df['A']
# Filtrar filas donde A > 1 filtered = df[df['A'] > 1]
```

Estas técnicas son esenciales para preparar y explorar datos antes de análisis más avanzados.

Aplicación práctica en ciencia de datos

En proyectos reales de ciencia de datos, la eficiencia en la manipulación de datos es clave para manejar grandes volúmenes y obtener resultados rápidos. NumPy proporciona la base para cálculos numéricos rápidos, mientras que Pandas ofrece estructuras intuitivas para trabajar con datos heterogéneos y etiquetados.

Por ejemplo, al analizar datos de ventas, puedes usar Pandas para cargar y filtrar registros de interés, y NumPy para realizar cálculos estadísticos o transformaciones vectorizadas que optimicen el rendimiento.

Configurar un entorno adecuado en Jupyter Notebook o Google Colab permite iterar rápidamente, visualizar resultados y documentar el proceso, facilitando la reproducibilidad y colaboración.

Este conocimiento es fundamental para preparar datos antes de aplicar técnicas de modelado o visualización, que se abordarán en unidades posteriores.

9 - Buenas prácticas en manipulación y gestión de tipos

Este contenido aborda estrategias profesionales para la gestión eficiente de tipos de datos en Pandas y NumPy, reforzando operaciones vectorizadas y transformaciones para optimizar memoria y rendimiento en análisis de datos.

Introducción

¿Sabías que una gestión adecuada de los tipos de datos puede acelerar significativamente tus análisis y reducir el consumo de memoria? En el contexto de Ciencia de Datos, donde trabajamos con grandes volúmenes de información, optimizar el manejo de tipos es clave para la eficiencia y escalabilidad de nuestros procesos.

En esta unidad, reforzaremos las operaciones vectorizadas que ya conoces y exploraremos buenas prácticas para manipular tipos de datos en Pandas y NumPy. Aprenderás a aplicar estrategias como el uso de tipos categóricos y downcasting para mejorar el rendimiento de tus DataFrames, además de cuándo y cómo convertir entre estructuras de Pandas y arrays de NumPy para aprovechar sus fortalezas.



Gestión eficiente de tipos de datos en Pandas y NumPy

Importancia del manejo de tipos

El tipo de dato (dtype) en Pandas y NumPy no solo define la naturaleza de la información, sino que impacta directamente en el uso de memoria y la velocidad de procesamiento. Por ejemplo, un entero almacenado como int64 consume más memoria que uno en int8.

Tip: Siempre que sea posible, utiliza el tipo más pequeño que represente tus datos sin pérdida de información.

Tipos categóricos en Pandas

Los tipos categóricos (category) son ideales para variables con un número limitado de valores únicos, como géneros, estados o categorías de productos. Al convertir una columna a tipo categórico:

- Se reduce el uso de memoria al almacenar internamente códigos numéricos en lugar de strings completos.
- Se mejora la velocidad en operaciones de filtrado y agrupamiento.

```
python
import pandas as pd
df['categoria'] = df['categoria'].astype('category')
```

Downcasting: reducción de uso de memoria

El downcasting consiste en convertir tipos numéricos a versiones más pequeñas cuando los valores lo permiten.

python

```
# Convertir float64 a float32
df['columna'] = pd.to_numeric(df['columna'], downcast='float')
# Convertir int64 a int8
df['columna'] = pd.to_numeric(df['columna'], downcast='integer')
```

Esta técnica es especialmente útil en datasets grandes para optimizar recursos.

Conversión entre Pandas y NumPy

Aunque Pandas ofrece gran funcionalidad, NumPy es más eficiente en operaciones numéricas vectorizadas y uso de memoria.

- Para cálculos intensivos, extraer arrays NumPy con `df.values` o `df.to_numpy()` puede acelerar procesos.
- Después de la manipulación, se puede volver a convertir a DataFrame para análisis y visualización.

Reforzamiento de transformaciones y operaciones vectorizadas

Recordemos que las operaciones vectorizadas permiten aplicar funciones a columnas o filas completas sin bucles explícitos, mejorando la eficiencia.

- Métodos como `.apply()`, `.assign()`, y operaciones aritméticas vectorizadas son fundamentales.

python

```
# Ejemplo: crear una nueva columna con transformación vectorizada
df = df.assign(nueva_columna = df['columna_existente'] * 2)
```

***Nota:** Evita usar bucles for para transformaciones sobre DataFrames, ya que son menos eficientes.*

Conocimiento clave: La gestión adecuada de tipos y el uso de operaciones vectorizadas son pilares para un análisis de datos eficiente y escalable.

Aplicación práctica en proyectos de Ciencia de Datos

En proyectos reales, los datasets suelen ser voluminosos y heterogéneos. La gestión eficiente de tipos permite:

- **Reducir el consumo de memoria:** Fundamental para trabajar en entornos con recursos limitados o para acelerar procesos en la nube.
- **Mejorar tiempos de ejecución:** Operaciones vectorizadas y tipos optimizados aceleran cálculos y transformaciones.

- **Preparar datos para modelado:** Algunos algoritmos requieren tipos específicos o se benefician de datos categóricos bien definidos.

Por ejemplo, en un análisis de ventas, convertir la columna "Región" a tipo categórico puede acelerar agrupamientos y reportes. Asimismo, downcasting de columnas numéricas evita desperdicio de memoria sin perder precisión.

Además, la conversión estratégica entre Pandas y NumPy permite aprovechar lo mejor de ambos mundos: facilidad de manipulación y eficiencia computacional.

Esta unidad te prepara para aplicar estas técnicas en tus propios proyectos, mejorando la calidad y rendimiento de tus pipelines de datos.

10 - Selección y filtrado: .loc, .iloc, máscaras y condiciones complejas

Ejercicio práctico para reforzar técnicas de selección y filtrado en DataFrames usando pandas, aplicando .loc, .iloc y máscaras booleanas con condiciones compuestas.

Ejercicio: Filtrado avanzado en DataFrames

Selección avanzada de datos en pandas: .loc, .iloc y máscaras booleanas

1. Introducción

En proyectos reales de análisis de datos, una de las tareas más frecuentes consiste en seleccionar subconjuntos específicos de información a partir de grandes volúmenes de datos.

Antes de modelar, visualizar o limpiar un dataset, el analista debe ser capaz de responder preguntas como:

- ¿Qué columnas necesito analizar?
- ¿Qué filas cumplen ciertas condiciones?
- ¿Cómo acceder a los datos de forma precisa y controlada?

La librería **pandas** provee distintos mecanismos de selección que, aunque pueden parecer similares, responden a **lógicas internas diferentes**. Comprender estas diferencias es fundamental para evitar errores silenciosos y resultados incorrectos.

Esta práctica tiene como objetivo **comprender el funcionamiento interno** de tres mecanismos clave:

- `.loc` → selección por etiquetas
- `.iloc` → selección por posición
- `mask` → filtrado booleano mediante condiciones lógicas

No se trata únicamente de “usar pandas”, sino de **entender qué ocurre internamente cuando se seleccionan datos**.

2. Contexto del dataset

El dataset simula un escenario realista de **e-commerce**, con información típica de un sistema transaccional:

- Identificación de órdenes
- Clientes
- Segmentos comerciales
- Categorías de producto
- Precios y cantidades
- Fechas
- Calificaciones
- Información logística
- Devoluciones

Este tipo de estructura es habitual en:

- análisis comercial
- reportes de ventas
- segmentación de clientes

- detección de anomalías
- dashboards ejecutivos

Trabajar sobre un dataset pequeño pero realista permite enfocarse en la **lógica del análisis**, sin perder claridad conceptual.

3. Selección de datos: por qué no existe una única forma

En pandas, **no todas las selecciones son iguales**, aunque visualmente puedan parecer similares.

Por ejemplo:

```
df[0:5]
```

y

```
df.loc[0:5]
```

no siempre producen el mismo resultado.

Esto ocurre porque pandas distingue claramente entre:

- **etiquetas (labels)**
- **posiciones (índices numéricos)**

Comprender esta diferencia es uno de los objetivos centrales de la práctica.

4. Selección con .loc: etiquetas

4.1 Concepto

El método `.loc` se basa en **etiquetas**, no en posiciones.

Esto implica que:

- Las columnas se seleccionan por **nombre**
- Las filas se seleccionan por **valor del índice**
- Los rangos son **inclusivos**

Ejemplo conceptual:

```
df.loc[2:6, "customer":"price"]
```

Incluye:

- fila 2 **hasta** fila 6
- columna "customer" **hasta** "price"

Ambos extremos están incluidos.

4.2 Qué deben hacer los estudiantes

En la práctica, los estudiantes deben **reconstruir manualmente el comportamiento de** `.loc`, lo que implica:

1. Obtener la lista de columnas del DataFrame.
2. Identificar la posición de la columna inicial y final.
3. Construir una lista intermedia de columnas.
4. Seleccionar filas por índice usando rangos inclusivos.
5. Retornar el subconjunto resultante.

Este ejercicio permite comprender que `.loc` no trabaja por posición, sino por etiquetas que luego pandas traduce internamente.

5. Selección con `.iloc`: posición

5.1 Concepto

El método `.iloc` trabaja exclusivamente por **posición numérica**:

- filas → números enteros
- columnas → números enteros
- el índice siempre comienza en 0

Ejemplo:

```
df.iloc[0:3, 1:4]
```

Selecciona:

- filas en posiciones 0, 1 y 2
- columnas en posiciones 1, 2 y 3

A diferencia de `.loc`, el slicing estándar de Python es exclusivo en el límite superior, por lo que en esta práctica se solicita adaptar el comportamiento para que sea inclusivo, forzando la comprensión del slicing.

5.2 Objetivo pedagógico

Implementar `.iloc` manualmente obliga a los estudiantes a:

- trabajar con slicing real de Python
- entender cómo pandas utiliza índices internos
- diferenciar claramente posición vs etiqueta

Este conocimiento es crítico para evitar errores comunes en análisis reales.

6. Filtrado por máscaras booleanas

6.1 Qué es una máscara

Una máscara booleana es una Serie de valores True y False que indica qué filas deben conservarse.

Ejemplo:

```
df["price"] > 150
```

produce algo como:

```
TrueFalseTrueFalse...
```

Luego, pandas filtra:

```
df[mask]
```

6.2 Uso de operadores lógicos

En pandas:

Operador lógico	Forma correcta
and	&
or	,
not	~

Además:

- **cada condición debe ir entre paréntesis**
- los strings deben ir entre comillas

Ejemplo correcto:

```
(df["segment"] == "Retail") & (df["price"] > 150)
```

Muchos errores frecuentes en análisis profesional provienen de no respetar estas reglas.

7. Preprocesamiento de condiciones

En la práctica, las condiciones se reciben como **texto**, por ejemplo:

```
segment==Retail and price>150
```

Para que Python pueda evaluarlas, deben transformarse a:

```
(segment == 'Retail') & (price > 150)
```

Por eso se incluye una función intermedia que:

1. Reemplaza operadores lógicos (and, or, not)
2. Detecta valores string y agrega comillas
3. Encierra cada comparación entre paréntesis

Este paso simula cómo funcionan internamente:

- motores de consulta
- filtros dinámicos
- sistemas de reporting
- dashboards interactivos

8. Evaluación de la máscara

Una vez procesada la condición:

1. Se construye un diccionario con las columnas del DataFrame.
2. Se evalúa la expresión usando `eval()`.
3. Se obtiene una Serie booleana.
4. Se filtran únicamente las filas verdaderas.

Este procedimiento replica el mecanismo interno que utilizan muchas herramientas analíticas modernas.

9. Valor de la práctica

Esta práctica te enseñará a:

- deje de usar pandas como “caja negra”
- comprenda qué ocurre detrás de `.loc`, `.iloc` y filtros
- fortalezca el pensamiento lógico
- adquiera base sólida para EDA real
- interprete correctamente errores comunes

No se busca memorizar sintaxis, sino comprender el razonamiento que gobierna la selección de datos.

10. Cierre conceptual

Al finalizar la práctica, el estudiante debería poder afirmar con claridad:

- `.loc` selecciona por **etiquetas**
- `.iloc` selecciona por **posición**
- las máscaras permiten **filtrado condicional**

- los operadores lógicos en pandas **no son los mismos que en Python puro**
- el uso correcto de paréntesis es obligatorio
- entender la selección es fundamental antes de cualquier modelo de Machine Learning

Dominar estos conceptos es un paso esencial para avanzar hacia análisis más complejos, validaciones, feature engineering y modelado predictivo.

11 - Transformaciones vectorizadas y operaciones en columnas

Ejercicio para aplicar transformaciones vectorizadas en columnas de un DataFrame simulado, usando operaciones vectorizadas y NumPy para optimizar cálculos.

Transformaciones vectorizadas y operaciones en columnas

1. Introducción

En análisis de datos, una de las tareas más frecuentes consiste en transformar columnas existentes para generar nuevas variables.

En contextos reales, rara vez trabajamos con los datos “tal como vienen”. Generalmente necesitamos:

- Calcular métricas derivadas
- Ajustar valores (descuentos, impuestos, tasas)
- Normalizar variables
- Crear indicadores
- Transformar formatos

En esta unidad aprenderemos a realizar estas transformaciones de forma eficiente y profesional, utilizando operaciones vectorizadas en pandas y NumPy.

2. ¿Qué significa "vectorizado"?

Una operación vectorizada es aquella que se aplica sobre una columna completa al mismo tiempo, sin necesidad de recorrer fila por fila con un bucle for.

En lugar de hacer esto (incorrecto en análisis profesional):

```
for i in range(len(df)):    df["total"][i] = df["quantity"][i] * df["price"][i]
```

Utilizamos:

```
df["total"] = df["quantity"] * df["price"]
```

La segunda opción es:

- Más rápida
- Más clara
- Más segura
- Más legible
- Escalable a millones de filas

Las operaciones vectorizadas están optimizadas en bajo nivel (C/NumPy), lo que permite alto rendimiento.

3. ¿Por qué evitar bucles explícitos?

En Data Science trabajamos con datasets grandes.

Un bucle explícito:

- Es lento
- Es propenso a errores
- No aprovecha la optimización interna de pandas
- Genera código menos limpio

Las operaciones vectorizadas permiten que el código sea:

- Declarativo

- Compacto
- Más cercano al lenguaje matemático
- Más mantenible

En términos profesionales, escribir código vectorizado es una buena práctica obligatoria.

4. Contexto del ejercicio

Simularemos un dataset de ventas con las siguientes columnas:

- `product_name`
- `quantity`
- `unit_price`

Queremos:

1. Calcular el total bruto por venta
2. Aplicar un descuento condicional
3. Generar el precio final
4. Crear indicadores derivados

Este escenario es completamente realista en entornos de e-commerce o retail.

5. Operaciones vectorizadas básicas

5.1 Operaciones aritméticas entre columnas

Las columnas de pandas se comportan como vectores matemáticos.

Podemos hacer:

- Sumas
- Restas

- Multiplicaciones
- Divisiones
- Potencias

Ejemplo conceptual:

```
df["total"] = df["quantity"] * df["unit_price"]
```

Cada fila realiza el cálculo correspondiente sin necesidad de bucles.

5.2 Operaciones con escalares

También podemos operar contra un valor fijo:

```
df["price_with_tax"] = df["unit_price"] * 1.21
```

Esto aplica el 21% a toda la columna automáticamente.

5.3 Operaciones condicionales vectorizadas

En lugar de usar if dentro de un loop, usamos:

- `np.where()`
- máscaras booleanas

Ejemplo:

```
df["discount"] = np.where(df["quantity"] > 10, 0.10, 0)
```

Esto significa:

- Si la cantidad es mayor a 10 → descuento 10%
- Caso contrario → 0%

Todo aplicado en una sola operación vectorizada.

6. Máscaras booleanas

Una máscara es una condición que devuelve True o False para cada fila.

Ejemplo:

```
mask = df["quantity"] > 5
```

Esto genera una serie booleana.

Podemos usarla para:

- Filtrar datos
- Asignar valores condicionales
- Crear nuevas columnas

Ejemplo:

```
df.loc[df["quantity"] > 5, "bulk_order"] = 1
```

7. Encadenamiento de operaciones

Una de las ventajas del enfoque vectorizado es que podemos encadenar transformaciones:

```
df["final_price"] = df["quantity"] * df["unit_price"] * (1 - df["discount"])
```

Esto permite expresar lógica de negocio de forma compacta y clara.

8. ¿Cuándo usar .apply()?

.apply() permite ejecutar una función personalizada sobre cada fila o cada elemento.

Sin embargo:

- Es más lento que una operación vectorizada pura
- Debe usarse solo cuando no exista alternativa vectorizada

Ejemplo adecuado de uso:

```
def category_label(price):
    if price < 50:
        return "Low"
    elif price < 200:
        return "Medium"
    else:
        return "High"
df["price_category"] = df["final_price"].apply(category_label)
```

Regla práctica:

Si puedes resolverlo con operaciones aritméticas o `np.where()`, no uses `.apply()`.

9. Beneficios de las transformaciones vectorizadas

- ✓ Mayor eficiencia
- ✓ Código más limpio
- ✓ Mejor rendimiento
- ✓ Escalabilidad
- ✓ Menor probabilidad de errores
- ✓ Mejores prácticas profesionales

En entornos reales con millones de registros, esta diferencia es crítica.

12. Principios de visualización y Matplotlib

Este contenido introduce los principios fundamentales para crear visualizaciones analíticas claras y efectivas, y presenta Matplotlib como la herramienta base para gráficos estáticos en Python. Se abordan tipos de gráficos comunes, criterios para su selección y buenas prácticas de diseño para comunicar datos con precisión y claridad.

¿Alguna vez te has preguntado por qué algunos gráficos comunican información de manera clara y otros no? En el análisis de datos, la visualización es la herramienta clave para transformar números en historias comprensibles y decisiones informadas. En esta unidad, exploraremos los principios fundamentales que garantizan que tus gráficos sean claros, precisos y útiles para el análisis.

Además, conoceremos Matplotlib, la biblioteca base en Python para crear gráficos estáticos, que te permitirá plasmar visualmente tus datos con flexibilidad y control. Aprenderás a elegir el tipo de gráfico adecuado según la pregunta analítica que quieras responder, y a aplicar buenas prácticas para que tus visualizaciones sean profesionales y efectivas.

Este conocimiento es esencial para avanzar en la construcción de análisis visuales integrados y prepararás la base para explorar visualizaciones interactivas en unidades posteriores.

Principios fundamentales de visualización analítica

Para comunicar datos de forma efectiva, un gráfico debe ser **claro**, **preciso** y **adecuado al contexto**. Estos son algunos principios clave:

- **Claridad:** El gráfico debe evitar elementos innecesarios que distraigan. Cada componente debe tener un propósito.
- **Escala adecuada:** Elegir escalas que representen fielmente los datos sin distorsiones.
- **Leyendas y etiquetas:** Deben ser descriptivas y legibles para facilitar la interpretación.
- **Selección del tipo de gráfico:** Depende de la naturaleza de los datos y la pregunta analítica.

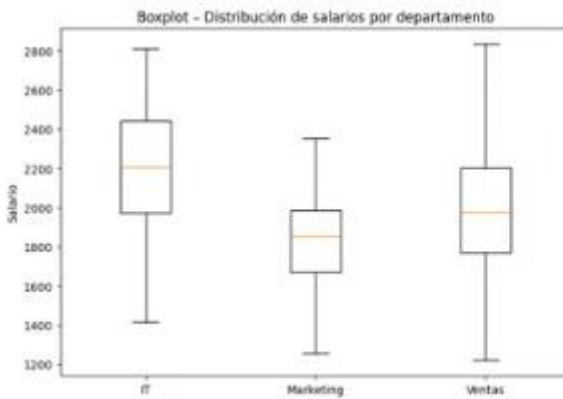
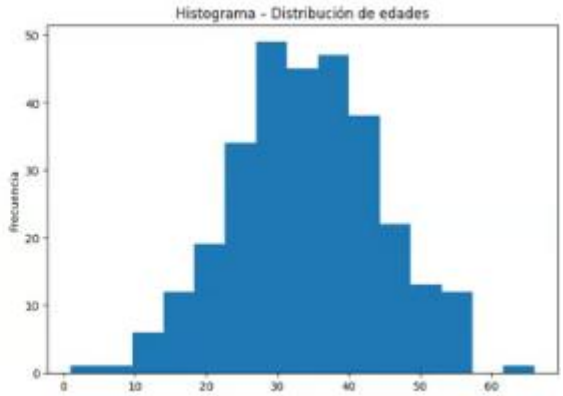
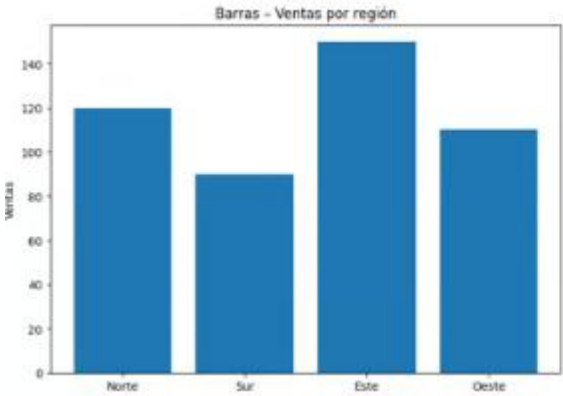
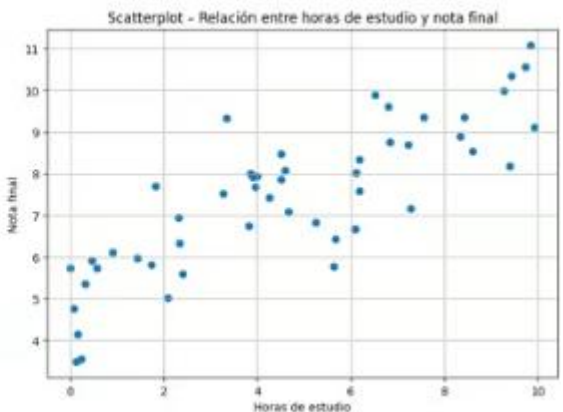
Tipos de gráficos estáticos en Matplotlib

Matplotlib permite crear diversos gráficos, cada uno útil para diferentes análisis:

Tipo de gráfico	Uso principal	Ejemplo de aplicación
Gráfico de líneas	Mostrar tendencias en series temporales	Evolución de ventas mensuales
Scatterplot (dispersión)	Visualizar relación entre dos variables numéricas	Correlación entre horas de estudio y nota final
Barras	Comparar cantidades entre categorías	Ventas por región
Histograma	Distribución de una variable numérica	Distribución de edades en una muestra

Tipo de gráfico	Uso principal	Ejemplo de aplicación
Boxplot (diagrama de caja)	Resumen estadístico y detección de outliers	Análisis de salarios por departamento
Pie chart (gráfico de pastel)	Proporciones relativas en un conjunto	Porcentaje de usuarios por dispositivo

***Nota:** Aunque los pie charts son populares, se recomienda usarlos con precaución, ya que pueden dificultar comparaciones precisas.*



Introducción a Matplotlib y su API

Matplotlib es la biblioteca estándar para gráficos estáticos en Python. Su estructura básica incluye:

- **Figura (figure):** Contenedor principal del gráfico.
- **Ejes (axes):** Área donde se dibuja el gráfico.
- **Funciones de trazado:** Métodos para crear diferentes tipos de gráficos, como `plot()`, `scatter()`, `bar()`, `hist()`, `boxplot()`, `pie()`.

Ejemplo básico para un gráfico de líneas:

```
python
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 17, 20]
plt.plot(x, y, marker='o', linestyle='-', color='b')
plt.title('Ejemplo de gráfico de líneas')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.grid(True)
plt.show()
```

Buenas prácticas en Matplotlib

- **Etiquetas claras:** Siempre incluir títulos, etiquetas de ejes y leyendas si hay múltiples series.
- **Colores y estilos:** Usar colores contrastantes y estilos consistentes para facilitar la lectura.
- **Escalas adecuadas:** Ajustar límites y escalas para evitar distorsiones.
- **Simplicidad:** Evitar elementos decorativos innecesarios que no aporten información.

Reflexión: ¿Qué tipo de gráfico usarías para mostrar la distribución de edades en un conjunto de datos?
¿Y para comparar ventas entre diferentes regiones?

En la práctica profesional de ciencia de datos, la visualización es una herramienta indispensable para explorar, comunicar y validar resultados. Por ejemplo, un analista que presenta un informe de ventas mensuales puede usar un gráfico de líneas para mostrar tendencias, mientras que un equipo de marketing puede preferir un gráfico de barras para comparar el rendimiento entre campañas.

Matplotlib, por su flexibilidad y amplia adopción, es la base para crear estos gráficos estáticos que luego pueden integrarse en reportes, dashboards o notebooks interactivos. Entender cómo seleccionar y personalizar estos gráficos según el contexto analítico es clave para transmitir insights con impacto.

Esta unidad prepara el terreno para que puedas construir visualizaciones claras y profesionales, que serán la base para incorporar interactividad y storytelling visual en módulos posteriores.

13. Matplotlib: Líneas, scatter, barras, histogramas y boxplots

Ejercicio para reforzar la creación y personalización de gráficos estáticos con Matplotlib, incluyendo líneas, scatter, barras, histogramas y boxplots. Se evaluará la capacidad para interpretar datos y generar visualizaciones adecuadas, además de guardar las figuras para su uso en reportes.

Ejercicio: Visualización con Matplotlib

1. La visualización como herramienta en Data Science

En un proyecto de Data Science, la visualización cumple un rol central en todas las etapas del análisis:

- **Exploración inicial de datos (EDA)**
- **Comprensión del comportamiento de variables numéricas**
- **Detección de patrones, dispersiones y valores atípicos**
- **Comunicación de resultados a equipos de negocio**

A diferencia de los modelos predictivos, las visualizaciones permiten **interpretar los datos de forma inmediata**, incluso para perfiles no técnicos.

Por este motivo, todo profesional en Data Science debe dominar las herramientas básicas de visualización antes de avanzar hacia librerías de mayor nivel como Seaborn o Plotly.

2. Matplotlib: la base del ecosistema de visualización en Python

Matplotlib es la librería de visualización fundamental del ecosistema científico de Python.

Muchas librerías modernas (Seaborn, pandas.plot, scikit-learn, etc.) están construidas internamente sobre Matplotlib.

Su dominio permite:

- Control total del gráfico

- Comprensión real de ejes, figuras y objetos
- Capacidad de personalizar visualizaciones para reportes técnicos
- Exportar gráficos para informes, notebooks o presentaciones

En esta práctica se utiliza el módulo:

```
import matplotlib.pyplot as plt
```

El cual ofrece una interfaz simple para crear gráficos estáticos.

3. Contexto de negocio del ejercicio

El dataset simulado representa un **escenario real de e-commerce**, donde cada fila corresponde a una orden de compra.

Entre las variables principales se encuentra:

- `sales_amount`: monto total de la orden
- `quantity`: cantidad de productos
- `category`: categoría del producto
- `order_date`: fecha de compra

El objetivo del análisis es **visualizar el comportamiento de las ventas individuales**, lo que permite responder preguntas como:

- ¿Cómo varían los montos entre órdenes?
 - ¿Existen valores extremos?
 - ¿Cómo se distribuyen las ventas?
 - ¿Qué tan dispersos están los datos?
-

4. Estructura general de una visualización en Matplotlib

Toda visualización básica en Matplotlib sigue una secuencia lógica:

1. **Crear la figura**
2. **Generar el gráfico**
3. **Agregar título**
4. **Etiquetar ejes**
5. **Guardar el archivo**
6. **Mostrar la figura**

Ejemplo conceptual:

```
plt.figure()plt.plot(datos)plt.title("Título")plt.xlabel("X")plt.ylabel("Y")plt.savefig("grafico.png")plt.show()
```

Comprender este flujo es fundamental, ya que se repite en todos los tipos de gráficos.

5. Gráfico de líneas (Line Plot)

¿Qué representa?

El gráfico de líneas se utiliza para analizar:

- Evoluciones
- Tendencias
- Cambios secuenciales

Aunque suele emplearse con series temporales, también puede utilizarse para observar el comportamiento ordenado de una variable.

En esta práctica

Se grafica:

- **Eje X:** índice del registro
- **Eje Y:** monto de venta (`sales_amount`)

Esto permite observar la variabilidad entre órdenes consecutivas.

¿Por qué es importante?

Permite detectar:

- picos de venta
 - valores inusuales
 - cambios abruptos
-

6. Gráfico de dispersión (Scatter Plot)

¿Qué representa?

El gráfico scatter muestra **la relación entre dos variables numéricas** o la dispersión de una variable respecto a otra.

Cada punto representa una observación individual.

En esta práctica

Se utiliza el mismo conjunto de datos que en el gráfico de líneas, pero representado como puntos independientes.

Esto permite analizar:

- concentración de valores
- dispersión
- presencia de outliers

A diferencia del gráfico de líneas, **no existe conexión entre observaciones**.

7. Gráfico de barras

¿Qué representa?

El gráfico de barras es ideal para:

- comparar magnitudes
- visualizar valores individuales

- mostrar conteos o totales

En esta práctica

Se construye un gráfico de barras donde:

- cada barra representa una orden
- la altura representa el monto de venta

Este tipo de gráfico permite identificar rápidamente órdenes de mayor o menor valor.

8. Histograma

¿Qué representa?

El histograma analiza la **distribución de una variable numérica**.

Agrupar los datos en intervalos llamados *bins*.

Conceptos clave

- Un bin representa un rango de valores
- La altura indica cuántas observaciones caen en ese rango

En esta práctica

Se solicita explícitamente:

- **10 bins**

Esto permite observar:

- si la distribución es simétrica o sesgada
- concentración de ventas
- rangos más frecuentes

El histograma es una herramienta central en el análisis exploratorio.

9. Boxplot

¿Qué representa?

El boxplot resume estadísticamente una variable numérica mediante:

- mediana
- cuartil 1 (Q1)
- cuartil 3 (Q3)
- rango intercuartílico
- valores atípicos (outliers)

¿Por qué es clave en Data Science?

Permite detectar rápidamente:

- dispersión
- asimetría
- presencia de outliers
- rango típico de los datos

En análisis reales, el boxplot suele ser uno de los primeros gráficos utilizados.

10. Guardado de figuras

En contextos profesionales, los gráficos no solo se visualizan, sino que se **exportan** para:

- reportes
- dashboards
- presentaciones
- documentación técnica

Por eso, en la práctica se exige:

```
plt.savefig("nombre.png")
```

Y luego:

```
plt.show()
```

Es importante comprender que:

- `savefig()` guarda el archivo
- `show()` muestra el gráfico en el notebook

Son acciones distintas.

Recomendaciones

- Usa `matplotlib.pyplot` para crear y guardar los gráficos.
- Personaliza cada gráfico con `title()`, `xlabel()`, `ylabel()`.
- Usa `plt.savefig('nombre.png')` para guardar la figura.
- Limpia la figura con `plt.clf()` o crea nuevas figuras para evitar sobreposición.

14. Plotly y Visualizaciones Interactivas

Ejercicio para crear gráficos interactivos usando Plotly en Python. Se busca que el estudiante implemente una función que genere un gráfico de barras interactivo a partir de datos de entrada, y que pueda exportar la visualización a un archivo HTML para su presentación y exploración.

Visualizaciones interactivas con Plotly para análisis de negocio

1. Introducción

En los proyectos de **Data Science orientados a negocio**, el análisis no finaliza cuando se obtienen métricas o agregaciones numéricas.

El verdadero valor aparece cuando esos resultados pueden **comunicarse de forma clara, visual e interactiva** a perfiles no técnicos como gerentes, analistas de negocio o equipos ejecutivos.

Las visualizaciones permiten:

- Identificar patrones rápidamente
- Comparar categorías o segmentos
- Detectar concentraciones o desequilibrios
- Facilitar la toma de decisiones basada en datos

En esta práctica se trabaja con **Plotly**, una de las librerías más utilizadas en entornos profesionales para la creación de **gráficos interactivos**, dashboards y reportes exportables a HTML.

2. Contexto del problema

Una empresa de **e-commerce** registra información de sus órdenes de venta:

- categoría del producto
- cantidad
- monto de la venta
- fecha

A partir de estos datos, el área de negocio necesita responder preguntas como:

- ¿Qué categorías generan mayor facturación?
- ¿Cuál es la participación relativa de cada categoría?
- ¿Cómo se distribuyen las ventas dentro del portafolio de productos?

Para ello, se construye un dataset agregado por categoría, que resume:

- **Ventas totales** (`sales_amount`)
- **Cantidad de órdenes** (`orders_count`)
- **Valor promedio por orden** (`avg_order_value`)

Este dataset será la base para las visualizaciones.

3. ¿Por qué usar Plotly?

A diferencia de librerías estáticas como Matplotlib o Seaborn, Plotly permite:

- Interacción con el mouse (hover, zoom, selección)
- Visualización dinámica en notebooks
- Exportación directa a archivos HTML
- Integración con dashboards y aplicaciones web

Esto lo convierte en una herramienta ideal para:

- Reporting ejecutivo
 - Presentaciones interactivas
 - Análisis exploratorio avanzado
 - Productos de datos (data products)
-

4. Tipos de visualizaciones trabajadas

En esta práctica se implementan **tres tipos de gráficos**, cada uno con un objetivo analítico distinto.

4.1 Gráfico de barras interactivo

¿Qué representa?

El gráfico de barras se utiliza para **comparar magnitudes entre categorías**.

En este caso permite observar:

- Qué categorías venden más
- Diferencias absolutas de facturación

- Ranking de desempeño

¿Por qué es importante?

Es el gráfico más utilizado en análisis de negocio porque:

- Facilita comparaciones directas
- Es intuitivo para cualquier audiencia
- Permite ordenar de mayor a menor

Implementación conceptual

Para construirlo se requiere:

1. Ordenar las categorías por ventas
2. Definir:
 - eje X $\rightarrow$ categoría
 - eje Y $\rightarrow$ ventas totales
3. Agregar etiquetas y título
4. Exportar el gráfico a HTML

Plotly Express simplifica este proceso mediante `px.bar()`.

4.2 Gráfico Pie / Donut

¿Qué representa?

El gráfico pie muestra **la participación porcentual de cada categoría sobre el total**.

Permite responder preguntas como:

- ¿Qué porcentaje del negocio depende de una categoría?
- ¿Existe concentración de ventas?
- ¿Hay categorías marginales?

Donut vs Pie

El gráfico tipo **donut** es una variante del pie que:

- Mejora la legibilidad
- Permite centrar la atención en el conjunto
- Es más utilizado en dashboards modernos

Implementación conceptual

Para construirlo:

1. Se usan las ventas totales como valores
2. Las categorías actúan como etiquetas
3. Se configura el parámetro `hole` para generar el donut
4. Se exporta como archivo HTML

En Plotly se implementa mediante `px.pie()`.

4.3 Treemap

¿Qué representa?

El treemap es una visualización **jerárquica basada en áreas**.

Cada rectángulo representa una categoría y su tamaño es proporcional al valor asociado (ventas).

¿Qué ventajas tiene?

- Permite detectar rápidamente concentraciones
- Combina comparación y proporción
- Muy utilizada en dashboards ejecutivos

Es ideal cuando:

- Hay muchas categorías
- Se quiere analizar distribución del negocio
- Se busca impacto visual

Implementación conceptual

Para el treemap se requiere:

1. Definir la jerarquía (en este caso, una sola capa: categoría)
2. Asignar las ventas como tamaño
3. Configurar etiquetas y tooltips
4. Exportar el gráfico a HTML

En Plotly se utiliza `px.treemap()`.

5. Exportación a HTML

Un aspecto clave de esta práctica es la **exportación de los gráficos**.

El método:

```
fig.write_html("archivo.html")
```

permite:

- Abrir el gráfico en cualquier navegador
- Compartirlo sin necesidad de Python
- Integrarlo en reportes o dashboards

Esto transforma la visualización en un **producto de datos reutilizable**, no solo en un resultado del notebook.

6. Flujo general de la práctica

La práctica sigue el siguiente flujo lógico:

1. Generación del dataset (ya resuelto)
2. Agregación por categoría (ya resuelto)
3. Implementación de funciones para:
 - barras

- pie / donut
 - treemap
4. Exportación de cada gráfico a HTML
 5. Visualización interactiva en el notebook con `fig.show()`

Este enfoque reproduce el flujo real de un proyecto profesional:

datos → métricas → visualización → comunicación

7. Objetivos

Al finalizar la práctica, el estudiante será capaz de:

- Crear visualizaciones interactivas con Plotly Express
 - Interpretar métricas de negocio mediante gráficos
 - Diferenciar qué tipo de gráfico usar según el objetivo analítico
 - Exportar visualizaciones como archivos HTML
 - Construir reportes visuales reutilizables
-

8. Relación con el mundo profesional

Este tipo de práctica replica tareas reales de:

- Data Analyst
- Business Intelligence Analyst
- Data Scientist orientado a producto
- Analytics Engineer

En entornos reales, estas visualizaciones suelen formar parte de:

- Dashboards ejecutivos

- Informes mensuales
- Presentaciones a stakeholders
- Productos de analítica embebida

✓ Conclusión

Las visualizaciones interactivas no son solo una cuestión estética.

Son una **herramienta estratégica de comunicación de datos**.

Dominar Plotly permite transformar análisis técnicos en información clara, accionable y comprensible para la toma de decisiones empresariales.

15 - Storytelling y reporting: comunicar hallazgos

En esta unidad, reforzaremos el uso de Matplotlib para crear visualizaciones estáticas efectivas y aprenderemos a estructurar reportes claros y reproducibles que comuniquen hallazgos de manera profesional. Además, exploraremos formatos de entrega adecuados para compartir análisis, desde notebooks hasta dashboards interactivos.

Imagina que has realizado un análisis exhaustivo de un conjunto de datos complejo y has generado visualizaciones que revelan patrones interesantes. ¿Cómo aseguras que tu audiencia comprenda y valore tus hallazgos? La respuesta está en el **storytelling** y el **reporting** efectivos: comunicar datos no solo con números y gráficos, sino con una narrativa clara y reproducible.

En esta unidad, reforzaremos tus habilidades en Matplotlib para crear gráficos estáticos que comuniquen con precisión y profesionalismo. Además, aprenderás a estructurar reportes que integren visualizaciones y conclusiones de forma reproducible, y a elegir el formato de entrega más adecuado para tu público y contexto.



Reforzando la visualización con Matplotlib

Matplotlib es la biblioteca fundamental para crear gráficos estáticos en Python. Dominar su uso es esencial para comunicar análisis de datos de forma clara y profesional.

Tipos de gráficos comunes y su uso

Tipo de gráfico	Uso principal	Ejemplo de aplicación
Gráfico de líneas	Mostrar tendencias en series temporales	Evolución de ventas mensuales
Scatterplot (dispersión)	Visualizar relación entre dos variables	Correlación entre horas de estudio y nota final
Barras	Comparar categorías	Ventas por región
Histograma	Distribución de una variable continua	Distribución de edades en una muestra
Boxplot	Resumen estadístico y detección de outliers	Análisis de salarios por departamento
Pie chart	Proporciones relativas	Porcentaje de usuarios por dispositivo

Buenas prácticas en visualización

- **Claridad ante todo:** evita gráficos recargados o con demasiada información.
- **Etiquetas descriptivas:** títulos, ejes y leyendas deben ser claros y concisos.
- **Colores adecuados:** usa paletas que faciliten la interpretación y sean accesibles.
- **Consistencia:** mantén estilos similares para facilitar la comparación.

Nota: Aunque Matplotlib es potente, para gráficos interactivos o dashboards se recomiendan otras herramientas, que veremos en unidades posteriores.

Storytelling y estructura de reportes reproducibles

Un reporte efectivo debe combinar visualizaciones con una narrativa que guíe al lector a través del análisis.

Componentes clave de un reporte reproducible:

1. **Introducción:** contexto y objetivos del análisis.
2. **Metodología:** descripción de los datos y procesos usados.
3. **Visualizaciones:** gráficos claros y bien explicados.
4. **Conclusiones:** hallazgos principales y recomendaciones.
5. **Código y datos:** para garantizar reproducibilidad.

Plantillas y checklist de reproducibilidad

- Uso de notebooks Jupyter o Google Colab para integrar código, visualizaciones y texto.
- Documentar cada paso con comentarios y explicaciones.
- Guardar versiones y dependencias para replicar el entorno.

Formatos de entrega

- **Notebooks:** ideales para análisis exploratorio y documentación técnica.
- **HTML/PDF:** para reportes estáticos y distribución formal.
- **Dashboards:** para visualizaciones interactivas y monitoreo en tiempo real.

La elección depende del público objetivo y el propósito del reporte.

En la práctica profesional de ciencia de datos, comunicar resultados es tan importante como realizar el análisis. Un reporte bien estructurado y visualizaciones claras facilitan la toma de decisiones y la colaboración con equipos multidisciplinarios.

Por ejemplo, un analista en una empresa de retail puede usar Matplotlib para crear gráficos que muestren tendencias de ventas y luego preparar un reporte reproducible en un notebook que incluya código y visualizaciones. Este reporte puede compartirse como PDF para la gerencia o como notebook para el equipo técnico.

Además, la reproducibilidad es fundamental para validar resultados y permitir que otros repliquen el análisis o lo extiendan. Por ello, integrar buenas prácticas de documentación y formatos adecuados es una habilidad clave para el científico de datos moderno.

16 - Storytelling y reporting: De la visualización al mensaje estratégico

Ejercicio para construir un pipeline con Scikit-Learn que incluya imputación, escalado y entrenamiento de un modelo de regresión lineal. Se evaluará la capacidad para integrar preprocesamiento y modelado en un flujo reproducible y eficiente.

Introducción

En la primera parte del módulo reforzamos el uso técnico de Matplotlib y la estructura básica de reportes reproducibles. Sin embargo, una visualización correcta no garantiza una comunicación efectiva.

En esta segunda parte profundizaremos en cómo transformar resultados analíticos en mensajes estratégicos, adaptados al público, alineados con objetivos de negocio y estructurados bajo principios de comunicación clara.

El foco ya no está solo en **cómo graficar**, sino en **cómo pensar la comunicación del análisis**.

Pensamiento narrativo en ciencia de datos

Del análisis al insight

Un gráfico muestra datos.

Un insight explica lo que esos datos significan.

Un buen reporte conecta ese insight con una decisión.

La diferencia entre informar y comunicar radica en la interpretación y el contexto.

Ejemplo:

- Informar: “Las ventas crecieron 12% en el último trimestre.”
- Comunicar: “El crecimiento del 12% está impulsado principalmente por la región Norte, lo que sugiere que replicar la estrategia comercial en otras regiones podría aumentar el revenue anual.”

El storytelling en data science implica:

1. Identificar el hallazgo central.
 2. Explicar por qué es relevante.
 3. Relacionarlo con el objetivo del análisis.
 4. Traducirlo en implicancias o recomendaciones.
-

Estructura narrativa profesional para reportes analíticos

Una estructura sólida evita reportes desorganizados o difíciles de interpretar.

Modelo recomendado para reportes ejecutivos y técnicos

1. Planteamiento del problema

- ¿Qué pregunta estamos respondiendo?

- ¿Cuál es el objetivo del análisis?
- ¿Qué decisión se quiere apoyar?

2. Contexto de negocio

- Marco en el que se realiza el análisis.
- Restricciones o supuestos relevantes.
- Métricas clave.

3. Metodología (nivel adecuado al público)

- Fuente de datos.
- Transformaciones principales.
- Técnicas utilizadas.
- Supuestos relevantes.

4. Resultados clave

- Visualizaciones seleccionadas estratégicamente.
- Interpretación directa debajo de cada gráfico.
- Comparaciones claras.
- Identificación de patrones, anomalías o tendencias.

5. Implicancias y recomendaciones

- ¿Qué debería hacerse con esta información?
- ¿Qué acciones concretas se sugieren?
- ¿Qué riesgos existen?

6. Limitaciones

- Calidad de datos.
- Tamaño muestral.
- Variables no observadas.
- Supuestos del modelo.

La transparencia fortalece la credibilidad.

Adaptación del mensaje según la audiencia

No todos los públicos requieren el mismo nivel técnico.

Audiencia ejecutiva

- Enfoque en decisiones.
- Métricas estratégicas.
- Visualizaciones simples.
- Poca explicación técnica.
- Conclusiones claras y accionables.

Audiencia técnica

- Detalle metodológico.
- Justificación estadística.
- Código reproducible.
- Métricas de validación.

- Consideración de edge cases.

Audiencia mixta

- Equilibrio entre claridad conceptual y respaldo técnico.
- Uso de anexos técnicos opcionales.

La habilidad del científico de datos no es solo analizar, sino adaptar el nivel de profundidad.

Diseño visual estratégico

En esta etapa se pasa de “gráficos correctos” a “gráficos intencionales”.

Jerarquía visual

Un gráfico debe guiar la mirada hacia lo importante.

Elementos clave:

- Título informativo (no genérico).
- Subtítulo contextual.
- Anotaciones estratégicas.
- Destacar el dato relevante.
- Minimizar elementos decorativos innecesarios.

Ejemplo:

En lugar de:

“Ventas por mes”

Preferir:

“Las ventas muestran una tendencia creciente sostenida desde Q2 2023”

El título ya comunica el insight.

Simplificación consciente

Menos es más.

Evitar:

- Exceso de colores.
- Grillas pesadas.
- Ejes innecesarios.
- Leyendas redundantes.
- Etiquetas ilegibles.

La simplicidad mejora la comprensión.

Visualizaciones comparativas y narrativa secuencial

Un solo gráfico rara vez cuenta toda la historia.

Existen tres formas comunes de narrativa visual:

1. Comparación temporal

Mostrar evolución en el tiempo.

2. Comparación categórica

Contrastar segmentos (regiones, productos, cohortes).

3. Descomposición

Explicar cómo una métrica global se compone de subcomponentes.

Un buen reporte organiza los gráficos en secuencia lógica:

1. Panorama general.
 2. Segmentación.
 3. Profundización.
 4. Conclusión.
-

Storytelling basado en hipótesis

Un enfoque profesional consiste en estructurar el análisis en torno a hipótesis.

Ejemplo:

Hipótesis:

“El churn aumenta en usuarios con baja frecuencia de uso.”

El reporte debe:

1. Mostrar la métrica global.
2. Segmentar por frecuencia.
3. Validar o refutar la hipótesis.
4. Extraer implicancias.

Esto transforma el análisis en un argumento lógico y estructurado.

Reproducibilidad avanzada

La reproducibilidad no es solo compartir código.

Incluye:

- Versionado de datos.
- Control de dependencias.
- Scripts ejecutables de principio a fin.
- Semillas aleatorias fijas.
- Documentación clara de cada paso.

Buenas prácticas:

- Separar carga, procesamiento y visualización.
- Evitar pasos manuales no documentados.
- Indicar versiones de librerías.
- Guardar outputs relevantes.

La reproducibilidad es una práctica ética y profesional.

Elección estratégica del formato de entrega

La forma en que se presenta el reporte impacta su recepción.

Notebook (Jupyter/Colab)

- Ideal para trabajo colaborativo técnico.
- Transparencia total.

- Excelente para documentación.

PDF ejecutivo

- Enfoque en síntesis.
- Ideal para dirección.
- Debe priorizar claridad y concisión.

HTML interactivo

- Permite exploración.
- Útil para stakeholders técnicos.

Dashboard

- Monitoreo continuo.
- Métricas actualizadas.
- Interactividad.

La elección depende de:

- Frecuencia de uso.
 - Nivel técnico del público.
 - Necesidad de actualización.
 - Toma de decisiones en tiempo real.
-

Errores comunes en storytelling de datos

1. Mostrar demasiados gráficos sin narrativa.
2. No contextualizar métricas.
3. Confundir correlación con causalidad.
4. Omitir limitaciones.
5. Usar visualizaciones incorrectas para el tipo de dato.
6. No destacar el insight principal.
7. Sobrecargar con terminología técnica innecesaria.

Un reporte no es un repositorio de gráficos, es un documento argumentativo.

Ética y responsabilidad en la comunicación

El científico de datos tiene responsabilidad en:

- No manipular escalas para exagerar resultados.
- No ocultar información relevante.
- No presentar correlaciones como causalidad.
- Comunicar incertidumbre cuando corresponde.

La credibilidad profesional depende de la integridad en la comunicación.

La visualización es una herramienta.

El storytelling es una competencia estratégica.

Un científico de datos profesional:

- Analiza rigurosamente.

- Visualiza con claridad.
- Interpreta con criterio.
- Comunica con intención.
- Recomienda con responsabilidad.

En esta segunda parte del módulo, el objetivo no es aprender más gráficos, sino aprender a transformar datos en decisiones.

Copíá y pegá esta celda de código en tu entorno de desarrollo para ver el paso a paso de la ejecución en la práctica.

¿Que vas a aprender?

Gráfico 1 — Tendencia temporal

Permite responder:

- Cómo evoluciona el negocio en el tiempo
- Detectar estacionalidad
- Detectar caídas o picos de ventas

Es el gráfico más usado en reporting ejecutivo.

Gráfico 2 — Comparación categórica

Permite responder:

- Qué categorías generan mayor revenue
- Cómo se distribuyen los ingresos del negocio

- Base para decisiones comerciales

Muy utilizado en dashboards.

Gráfico 3 — Distribución

Permite analizar:

- Concentración de ventas
- Presencia de órdenes pequeñas vs grandes
- Forma de la distribución

Clave en EDA profesional.

Gráfico 4 — Boxplot

Permite detectar:

- Outliers reales
- Dispersión del ticket promedio
- Asimetría en las ventas

Muy usado en análisis estadístico inicial.